

# 人工智能基础实验二

## 实验报告

实验题目：启发式解决八数码问题

专    业：人工智能

姓    名：马承乾

学    号：22920212204180

实验日期：2023.11.9

# 一. 实验题目

八数码问题的 A\*算法解决

# 二. 实验目的

熟悉 A\*算法的基本原理。  
理解八数码问题，并使用 A\*算法求解。  
实现并优化 A\*算法的数据结构和关键函数。

# 三. 实验方法

## ➤ 问题描述：

八数码问题是一个经典的拼图问题，通常使用 3x3 的数码板表示。数码板上的每个位置可以填充数字 1 到 8，另有一个位置为空格。初始状态下，这些数字被随机排列在数码板上。在本次实验中，我们的目标是通过移动数码，将初始乱序状态转化为目标状态。目标状态如下：

```
1 2 3
8  4
7 6 5
```

其中，空格用于移动相邻的数字。例如，可以通过将空格与相邻的数字交换位置来改变数码的排列顺序。我们的任务是使用 A\*算法找到从初始状态到目标状态的最短路径。

## ➤ A\*算法原理

A\*算法是一种启发式搜索算法，它维护一个优先级队列，每次选择代价最小的节点进行扩展。算法使用启发式估计函数（heuristic function）来估算从起始节点到目标节点的代价。在八数码问题中，启发式函数可以是当前状态到目标状态的曼哈顿距离或其他适当的估计。

## ➤ 数据结构和关键函数

🔗 My\_PriorityQueue 类： 自定义优先级队列，支持节点按照优先级排序。

```

class My_PriorityQueue(object):
> def __init__(self): ...
>
> def push(self, item, priority): ...
>
> def pop(self): ...
>
> def qsize(self): ...
>
> def empty(self): ...
>
> def exist_node(self, item):# 是否存在样子为item的样子的结点 ...
>
> def same_shape_node(self, item): ...
>
> def delete_node(self, item): ...

```

**push(item, priority):** 将节点按照优先级加入队列。

```

def push(self, item, priority):
    """
    队列由 (priority, index, item) 形式组成
    priority 增加 "-" 号是因为 heappush 默认是最小堆，我们这里不加负号，因为我们的优先级就是距离，距离越小优先级越高
    index 是为了当两个对象的优先级一致时，按照插入顺序排列
    """
    # heapq.heappush(self._queue, (-priority, self._index, item))
    heapq.heappush(self._queue, (priority, self._index, item))
    self._index += 1

```

**pop():** 弹出优先级最高的节点。

```

def pop(self):
    """
    弹出优先级最高的对象
    """
    return heapq.heappop(self._queue)[-1]

```

**exist\_node(item):** 检查队列中是否存在具有相同状态的节点。

```

def exist_node(self, item):# 是否存在样子为item的样子的结点
    for i in self._queue:
        if item.shape == i[-1].shape:
            return True
    return False

```

**same\_shape\_node(item):** 获取队列中具有相同状态的节点。

```

def same_shape_node(self, item):
    for i in self._queue:
        if item.shape == i[-1].shape:
            return i

```

**delete\_node(item):** 删除队列中具有相同状态的节点。

```
def delete_node(self, item):
    for i in self._queue:
        if item[-1].shape == i[-1].shape:
            del i
            break
```

🚦 Node 类： 表示八数码问题的节点。

```
class Node(object):
    def __init__(self, shape, h, parent, direction): ...

    def __repr__(self): ...

    # def count(self, shape):# 评估方案1
    #     global target
    #     num = 0
    #     for i in range(3):
    #         for j in range(3):
    #             if shape[i][j] != target[i][j]:
    #                 num += 1
    #     return num

    def count(self, shape):# 评估方案2 ...

    def position(self): ...
```

**\_\_init\_\_(shape, h, parent, direction):** 初始化节点，包括状态、启发式估计、父节点和移动方向。我们对其中变量做如下解释：

#### **f (综合代价函数值)：**

定义： f 表示从起始节点到当前节点的综合代价，即实际代价 g 和启发式函数值 h 的和。

在 A\*算法中，节点的选择是基于 f 值的，选择 f 最小的节点进行扩展。

表达式： 对于 Node 类中的 f，通常有如下表达式：

$$f(n) = g(n) + h(n)$$

其中，g(n)是从起始节点到节点 n 的实际代价，h(n) 是从节点 n 到目标节点的启发式函数值。

#### **h (启发式函数值)：**

定义： h 表示从当前节点到目标节点的估计代价，即启发式函数的值。启发式函数是一种用于估计最优解离目标有多远的函数。

在 Node 类中的实现： 在我的代码中，h 的计算是通过 count 函数实现的。具体而言，h 是 count 函数的返回值，该函数估算了当前状态到目标状态的代价，可以是曼哈顿距离或其他适当的估计。

```
def __init__(self, shape, h, parent, direction):
    self.shape = shape
    self.h = h
    self.g = self.count(shape)
    self.parent = parent
    self.direction = direction
```

**count(shape):** 估算当前状态到目标状态的代价。对两种评估方案，我们做如下解释：

评估方案 1:

这个评估方案使用的是曼哈顿距离（Manhattan Distance）的思想。它遍历当前状态和目标状态的每个位置，如果对应位置的数字不相等，就认为需要进行一次移动。因此，num 的值表示了当前状态与目标状态之间的不匹配程度，即需要移动的步数。

评估方案 2:

这个评估方案使用的是每个数字在当前状态和目标状态中的位置差的平方和。具体来说，对于每个数字，它计算了当前状态中该数字的位置与目标状态中该数字的位置的水平和垂直方向上的差的平方，然后累加到 num 中。这样的评估方式考虑了每个数字在当前状态和目标状态中的相对位置，而不仅仅是简单地计算不匹配的数字个数。

```
# def count(self, shape):# 评估方案1
#     global target
#     num = 0
#     for i in range(3):
#         for j in range(3):
#             if shape[i][j] != target[i][j]:
#                 num += 1
#     return num

def count(self, shape):# 评估方案2
    global target
    num = 0
    for i in range(1, 9):
        pos_i_1, pos_j_1 = -1, -1
        for x in range(3):
            for y in range(3):
                if shape[x][y] == str(i):
                    pos_i_1, pos_j_1 = x, y
        pos_i_2, pos_j_2 = -1, -1
        for x in range(3):
            for y in range(3):
                if target[x][y] == str(i):
                    pos_i_2, pos_j_2 = x, y
        num += pow(pos_i_1 - pos_i_2, 2) + pow(pos_j_1 - pos_j_2, 2)
    return num
```

**position():** 获取空白格的位置。

```
def position(self):
    for i in range(3):
        for j in range(3):
            if self.shape[i][j] == ' ':
                return [i, j]
```

## ➤ 实验步骤

步骤如下：

初始化起始节点，将其加入优先级队列。

使用 A\*算法进行状态扩展，直到找到目标状态或队列为空。

在每次扩展时，检查已访问节点，更新队列中相同状态节点的信息。  
输出解决方案的移动步骤和最终状态。

```
if __name__ == "__main__":
    node = Node(origin, 0, None, '')
    pq = My_PriorityQueue()
    pq.push(node, node.f + node.h)
    visited_nodes = My_PriorityQueue()
    flag = 0
    while pq.qsize() != 0:
        q_front = pq.pop()
        q_front_copy = copy.deepcopy(q_front)
        visited_nodes.push(q_front_copy, q_front_copy.f + q_front_copy.h) # 已访问
        if q_front.shape == target:
            Print(q_front_copy)
            flag = 1
            break
        if pq.exist_node(q_front_copy): # 这个样子已经存在，则观察当前结点和已经走过的节点的g+h谁大谁小
            visited_node = pq.same_shape_node(q_front_copy)
            if visited_node[-1].h + visited_node[-1].f < q_front.h + q_front.f: # 之前那个点更优
                continue # 抛弃当前点
            else: # 当前点更好，原来的点出队，当前点入队
                pq.delete_node(visited_node)
                pq.push(q_front_copy, q_front_copy.f + q_front_copy.h)
        position = q_front_copy.position()
        if position[1] < 2: # 右移...
        if position[1] > 0: # 左移...
        if position[0] < 2: # 下移...
        if position[0] > 0: # 上移...
    if flag == 0:
        print('无解决方案')
```

#### ➤ 优化策略

检查已访问节点：在每次扩展时，避免重复访问相同状态的节点。

```
break
if pq.exist_node(q_front_copy): # 这个样子已经存在，则观察当前结点和已经走过的节点的g+h谁大谁小
    visited_node = pq.same_shape_node(q_front_copy)
    if visited_node[-1].h + visited_node[-1].g < q_front.h + q_front.g: # 之前那个点更优
        continue # 抛弃当前点
    else: # 当前点更好，原来的点出队，当前点入队
        pq.delete_node(visited_node)
        pq.push(q_front_copy, q_front_copy.f + q_front_copy.h)
```

删除节点：当发现更优的路径到达相同状态节点时，删除队列中原有的节点，保留新的更优节点。

## 四、实验结果

🎨 展示成功解决八数码问题的例子，包括移动步骤和最终状态。

例子 1:

使用评估方案 1

```
target = [['1', '2', '3'],
          ['8', ' ', '4'],
          ['7', '6', '5']]
origin = [['1', '2', '3'],
          ['8', '4', '5'],
          ['7', ' ', '6']]
```

得到解决方案如下：

```
1 2 3
8 4 5
7 6
right
1 2 3
8 4 5
7 6
up
1 2 3
8 4
7 6 5
left
1 2 3
8 4
7 6 5
time spent is 0.003001 seconds
number of steps is 5
```

成功！

使用评估方案 2：

```
1 2 3
8 4 5
7 6
right
1 2 3
8 4 5
7 6
up
1 2 3
8 4
7 6 5
left
1 2 3
8 4
7 6 5
time spent is 0.003001 seconds
number of steps is 5
```

与使用评估方案 2 时相同，成功！

例子 2:

```
target = [['1', '2', '3'],
           ['8', ' ', '4'],
           ['7', '6', '5']]
origin = [['1', '6', '2'],
           ['8', ' ', '3'],
           ['7', '5', '4']]
```

使用评估方案 1:

```
1 6 2
8 3
7 5 4
up
1 2
8 6 3
7 5 4
right
1 2
8 6 3
7 5 4
down
1 2 3
8 6
7 5 4
down
1 2 3
8 6 4
7 5
left
1 2 3
8 6 4
7 5
up
1 2 3
8 4
7 6 5
time spent is 0.007999 seconds
number of steps is 24
```

使用评估方案 2:



```
1 6 2
8 3
7 5 4
up
1 2
8 6 3
7 5 4
right
1 2
8 6 3
7 5 4
down
1 2 3
8 6
7 5 4
down
1 2 3
8 6 4
7 5
left
1 2 3
8 6 4
7 5
up
1 2 3
8 4
7 6 5
time spent is 0.005957 seconds
number of steps is 7
```

🔗 讨论 A\*算法的空间复杂度，观察算法的性能。

A\*算法的空间复杂度主要取决于开放列表（Open List）和已访问节点列表的大小。让我来解释一下：

开放列表（Open List）： 这是一个优先级队列，用于存储待扩展的节点。在最坏情况下，开放列表可能包含所有生成的节点。因此，开放列表的空间复杂度为：

$$O(b^d),$$

其中 b 是分支因子，表示每个节点可以生成的子节点的数量，d 是解的深度。

已访问节点列表： 为了避免重复扩展相同的节点，通常需要维护一个已访问节点列表。在最坏情况下，已访问节点列表可能包含所有生成的节点。因此，已访问节点列表的空间复杂度也为：

$$O(b^d),$$

综合考虑，A\*算法的空间复杂度为：

$$O(b^d).$$

## 五. 实验分析

### 讨论算法性能

在解决八数码问题的 A\*算法中，启发式函数的选择以及一些优化策略对算法性能产生了重要影响。

启发式函数的选择涉及到权衡准确性和计算效率。评估方案 1 使用曼哈顿距离的思想，简单直观但可能在某些情况下估计不够准确。评估方案 2 考虑了数字的相对位置，提供了更准确的估计，但计算较为复杂。在实际应用中，需要通过实验和测试来确定哪种启发式函数更适合解决八数码问题，以达到提高算法性能的目的。

优化策略的选择也对算法性能起到关键作用。检查已访问节点和删除节点的策略有助于减少冗余计算和扩展相同节点的次数，提高搜索效率。使用优先级队列维护开放列表使得算法能够高效地选择最有希望的节点进行扩展。

然而，A\*算法也存在一些局限性。它对所选启发式函数的依赖性很高，如果选择的启发式函数不够准确，可能导致搜索路径不是最优解。此外，算法的空间复杂度可能较高，在处理大规模问题时可能面临资源受限的问题。算法对搜索空间规模的敏感性，以及在动态环境下的适应性相对有限，都是需要考虑的方面。

总体而言，A\*算法在八数码问题中展现了一定的效果，但在具体问题中需要综合考虑问题特性，进行实验和测试，以优化启发式函数的选择和调整优化策略，从而提高算法性能。

### 优化策略效果

在我的代码中，有几个优化策略旨在提高 A\*算法的效率：

**检查已访问节点：** 通过 `exist_node` 函数检查当前生成的节点是否已经存在于优先级队列中，以避免重复扩展相同的节点。这个策略有助于减少重复计算和扩展相同的节点，从而提高算法的效率。避免了不必要的重复工作，减小了搜索空间的大小。

**删除节点：** 通过 `delete_node` 函数删除已经存在于优先级队列中的相同状态的节点，以保留新的更优节点。这个策略的效果在于保持队列中相同状态节点的最优性，避免了低效的重复计算，同时保留了更有可能达到最优解的节点。

使用优先级队列：使用优先级队列来维护开放列表，确保每次选择具有最小综合代价函数值的节点进行扩展。优先级队列的使用使得 A\*算法能够高效地选择最有希望的节点进行扩展，有助于快速找到最优解。

这些优化策略主要体现在减少了冗余计算和扩展相同节点的次数，从而提高了 A\*算法的搜索效率。特别是在处理较大的搜索空间时，这些优化策略可以显著降低算法的时间和空间复杂度。需要注意的是，这些优化策略在不同问题和场景中的效果可能有所不同，因此优化策略的选择和调整应该根据具体情况进行。

### 启发式函数选择

在本次实验中，启发式函数的选择对 A\*算法的性能起着关键作用。存在两种不同的启发式函数选择方案，分别是评估方案 1 和评估方案 2。

评估方案 1 使用曼哈顿距离的思想，计算当前状态与目标状态之间数字位置的不匹配数量。这是一种简单直观的启发式函数，适用于八数码问题。它在评估问题状态时考虑了每个数字的不匹配情况，但在某些情况下可能估计不够准确。

评估方案 2 则更为复杂，考虑了每个数字在当前状态和目标状态中的位置差的平方和。这种启发式函数在一些情况下可能提供更准确的估计，特别是对于需要考虑数字相对位置的问题。然而，它的计算相对较复杂，可能在计算效率上略显耗费。

在实际应用中，启发式函数的选择需要在准确性和计算效率之间进行权衡。评估问题特性，考虑问题领域的需求，以及在实际应用中对不同启发式函数的实验和测试都是必要的步骤。对于八数码问题，可以通过实验和调整来确定哪种启发式函数更适合，从而提高 A 算法在解决问题上的性能。因此，启发式函数的选择和调整是 A 算法优化的重要方向之一。

### 算法的局限性

尽管 A\*算法在许多问题上表现出色，但它也存在一些局限性，其中一些局限性可能对特定情况下的性能产生负面影响。

A\*算法的性能高度依赖于所选的启发式函数。如果选择的启发式函数不够准确，可能导致算法搜索的路径不是最优解。在实际应用中，设计一个既准确又高效的启发式函数是一项具有挑战性的任务。同时，A\*算法的空间复杂度可能较高，特别是在处理大规模问题时。需要维护一个优先级队列用于存储待扩展的节点，以及一个已访问节点列表，这可能占用大量内存。在资源受限的环境下，这可能成为一个限制因素。

A\*算法对于搜索空间的规模非常敏感。在搜索空间非常庞大的情况下，A 算法可能会面临指数级增长的情况，导致计算时间增加。虽然一些优化策略可以缓解这个问题，但在极端情况下仍可能表现不佳。此外，A\*算法对于问题的可解性和解空间的结构假设较为乐观。在某些问题中，可能存在大量的局部最小值或者解的分支结构，使得 A 算法可能在搜索过程中陷入局部最小值，而无法找到全局最优解。

总体而言，虽然 A\*算法在许多情况下表现优异，但在面对一些具体问题时，需要谨慎考虑其局限性，并结合问题的特性选择合适的算法或进行适当的优化。

## 六. 附录

完整代码如下：

```
import copy
import heapq
from collections import deque
import time

class My_PriorityQueue(object):
    def __init__(self):
        self._queue = []
        self._index = 0

    def push(self, item, priority):
        """
        队列由 (priority, index, item) 形式组成
        priority 增加 "-" 号是因为 heappush 默认是最小堆，我们这里不加负号，
        因为我们的优先级就是距离，距离越小优先级越高
        index 是为了当两个对象的优先级一致时，按照插入顺序排列
        """
        # heapq.heappush(self._queue, (-priority, self._index, item))
        heapq.heappush(self._queue, (priority, self._index, item))
        self._index += 1

    def pop(self):
        """
        弹出优先级最高的对象
        """
        return heapq.heappop(self._queue)[-1]

    def qsize(self):
        return len(self._queue)

    def empty(self):
```

```

        return True if not self._queue else False

def exist_node(self, item):# 是否存在样子为 item 的样子的结点
    for i in self._queue:
        if item.shape == i[-1].shape:
            return True
    return False

def same_shape_node(self, item):
    for i in self._queue:
        if item.shape == i[-1].shape:
            return i

def delete_node(self, item):
    for i in self._queue:
        if item[-1].shape == i[-1].shape:
            del i
            break

class Node(object):
    def __init__(self, shape, h, parent, direction):
        self.shape = shape
        self.h = h
        self.g = self.count(shape)
        self.parent = parent
        self.direction = direction

    def __repr__(self):
        return self.direction

    # def count(self, shape):# 评估方案 1
    #     global target
    #     num = 0
    #     for i in range(3):
    #         for j in range(3):
    #             if shape[i][j] != target[i][j]:
    #                 num += 1
    #     return num

    def count(self, shape):# 评估方案 2
        global target
        num = 0
        for i in range(1, 9):
            pos_i_1, pos_j_1 = -1, -1

```

```

        for x in range(3):
            for y in range(3):
                if shape[x][y] == str(i):
                    pos_i_1, pos_j_1 = x, y
            pos_i_2, pos_j_2 = -1, -1
        for x in range(3):
            for y in range(3):
                if target[x][y] == str(i):
                    pos_i_2, pos_j_2 = x, y
        num += pow(pos_i_1 - pos_i_2, 2) + pow(pos_j_1 - pos_j_2, 2)
    return num

def position(self):
    for i in range(3):
        for j in range(3):
            if self.shape[i][j] == ' ':
                return [i, j]

def swap(t1, t2):
    temp = t1
    t1 = t2
    t2 = temp
    return t1, t2

def Print(node):# 打印
    dq = deque()
    while node is not None:
        dq.append(node)
        node = node.parent

    while len(dq):
        q_front = dq.pop()
        print(q_front.direction)
        for i in q_front.shape:
            for j in i:
                print(j, end=' ')
            print()

target = [['1', '2', '3'],
          ['8', ' ', '4'],
          ['7', '6', '5']]
origin = [['1', '6', '2'],
          ['8', ' ', '3'],
          ['7', '5', '4']]

```

```

if __name__ == "__main__":
    start_time = time.time()
    node = Node(origin, 0, None, '')
    pq = My_PriorityQueue()
    pq.push(node, node.g + node.h)
    visited_nodes = My_PriorityQueue()
    flag = 0
    num_step = 0
    while pq.qsize() != 0:
        num_step += 1
        q_front = pq.pop()
        q_front_copy = copy.deepcopy(q_front)
        visited_nodes.push(q_front_copy, q_front_copy.g +
q_front_copy.h)# 已访问
        if q_front.shape == target:
            Print(q_front_copy)
            flag = 1
            break
        if pq.exist_node(q_front_copy):# 这个样子已经存在，则观察当前结点和
已经走过的节点的 g+h 谁大谁小
            visited_node = pq.same_shape_node(q_front_copy)
            if visited_node[-1].h + visited_node[-1].g < q_front.h +
q_front.g:# 之前那个点更优
                continue# 抛弃当前点
            else:# 当前点更好，原来的点出队，当前点入队
                pq.delete_node(visited_node)
                pq.push(q_front_copy, q_front_copy.g + q_front_copy.h)
        position = q_front_copy.position()
        if position[1] < 2: # 右移
            shape = copy.deepcopy(q_front_copy.shape)
            shape[position[0]][position[1]],
shape[position[0]][position[1] + 1] = shape[position[0]][position[1] +
1], shape[position[0]][position[1]]
            node = Node(shape, q_front_copy.h + 1, q_front_copy,
'right')
            pq.push(node, node.g + node.h)
        if position[1] > 0: # 左移
            shape = copy.deepcopy(q_front_copy.shape)
            shape[position[0]][position[1]],
shape[position[0]][position[1] - 1] = shape[position[0]][position[1] -
1], shape[position[0]][position[1]]
            node = Node(shape, q_front_copy.h + 1, q_front_copy, 'left')
            pq.push(node, node.g + node.h)

```

```

        if position[0] < 2:# 下移
            shape = copy.deepcopy(q_front_copy.shape)
            shape[position[0] + 1][position[1]],
shape[position[0]][position[1]] = shape[position[0]][position[1]],
shape[position[0] + 1][position[1]]
            node = Node(shape, q_front_copy.h + 1, q_front_copy, 'down')
            pq.push(node, node.g + node.h)
        if position[0] > 0:# 上移
            shape = copy.deepcopy(q_front_copy.shape)
            shape[position[0] - 1][position[1]],
shape[position[0]][position[1]] = shape[position[0]][position[1]],
shape[position[0] - 1][position[1]]
            node = Node(shape, q_front_copy.h + 1, q_front_copy, 'up')
            pq.push(node, node.g + node.h)
    if flag == 0:
        print('no solution')
    end_time = time.time()
    print("time spent is %.6f seconds" % (end_time-start_time))
    print("number of steps is %d"%num_step)

```